NEXACHAIN AI

Eco Space, Business Tower, Newtown
Kolkata, West Bengal, India
info@nexachain.ai www.nexachain.ai

Nexa Chain

MERN Stack Developer Technical Assessment

Objective

Build the backend architecture, business logic, and frontend dashboard for an investment and referral-based platform using the MERN Stack (MongoDB, Express.js, React.js, Node.js).

Task 1: Database Schema Design

Design MongoDB (Mongoose) models for the following entities:

1. User

The user model should contain:

- Full name
- Email
- Mobile number
- Password (encrypted)
- Referral code
- Referred by (parent user)
- Wallet balance
- Total ROI earned
- Total level income earned
- Account status

2. Investment

Each investment should contain:

- User reference
- Investment amount
- Plan details
- Start date
- End date
- Daily ROI percentage
- Investment status (Active/Completed/Cancelled)

3. Referral / Level Income

Store referral earnings information:

- User who receives the income
- User who generated the income
- Referral level
- Income amount
- Date

4. ROI History

Track daily ROI generated from investments:

- User reference
- Investment reference
- ROI amount
- Date
- Status

Expected Deliverable

Create Mongoose schemas and model files for all the above entities with proper relationships and indexing.

Task 2: API Development

Create secure REST APIs for the following operations:

Authentication APIs

- User Registration
- User Login
- JWT Authentication

Investment APIs

- Create Investment
- View User Investments

Dashboard APIs

Return:

- Total Investments
- Total ROI Earned
- Total Level Income Earned
- Wallet Balance

Referral APIs

- Fetch Direct Referrals
- Fetch Complete Referral Tree

Requirements

- Use JWT authentication.
- Protect all private routes.
- Follow proper API structure and error handling.
- Optimize database queries for performance.

Expected Deliverable

Postman Collection or API Documentation with sample request and response data.

Task 3: Business Logic Implementation

Implement backend logic to calculate and distribute:

Daily ROI

For all active investments:

1. Calculate daily ROI.
2. Store ROI history.
3. Update user's wallet balance.

Referral / Level Income

1. Traverse referral hierarchy.
2. Calculate income for each level.
3. Credit earnings to eligible users.
4. Store transaction history.

Requirements

- Avoid duplicate calculations.
- Maintain transaction consistency.
- Use clean and scalable code structure.

Expected Deliverable

Service functions with proper documentation and comments.

Task 4: React Dashboard

Build a responsive React dashboard displaying:

Dashboard Cards

- Total Investments
- Daily ROI
- Total Level Income
- Wallet Balance

Tables

- Investment History
- ROI History
- Referral Income History

Referral Tree

Display referral hierarchy in nested format.

Additional Requirements

- Integrate APIs.
- Handle loading and error states.
- Use charts for visual representation of earnings.
- Follow good UI/UX practices.

Expected Deliverable

Working React dashboard with clean code structure.

Task 5: Cron Job / Scheduler

Implement an automated scheduler using node-cron.

Functionality

The scheduler should:

1. Run every day at 12:00 AM.
2. Process ROI calculations for active investments.
3. Update user balances.
4. Store ROI history records.

Important Requirement

The process must be idempotent:

- If the job runs twice accidentally, ROI should not be credited twice.

Expected Deliverable

Cron service implementation with proper safeguards against duplicate processing.

Submission Requirements

Submit:

1. GitHub Repository Link
2. Database Schema Files
3. API Source Code
4. Business Logic Implementation
5. React Dashboard
6. Cron Job Implementation
7. README File containing:
    - Project setup steps
    - Environment variables
    - API documentation
    - Assumptions made during development

Evaluation Criteria

Candidates will be evaluated based on:

- Database Design
- API Architecture
- Code Quality
- Security Practices
- Business Logic Accuracy
- Frontend UI/UX
- Performance Optimization
- Scalability
- Documentation Quality